AI Engineering and Claude Architect · Part I, Foundations

AI and ML Fundamentals, Lesson 1

What Learning From Data Means

Your fraud rule catches almost everything in January. By the following year, fraudsters have moved to smaller amounts and the same rule catches nothing. Nobody changed the code, and that is exactly the problem.

Module	AI01 of 13
Lesson	1 of 4
Prerequisites	None. Backend experience helps
Exam relevance	Supports only. The exam doesn't test ML theory, but this lesson explains the "probabilistic versus deterministic" rule that it tests repeatedly
Reading time	~35 minutes

Contents

1. What Learning From Data Means

Lesson 1 of 4 · AI and ML Fundamentals

What Learning From Data Means

Key idea

Machine learning replaces rules you write with rules inferred from examples. You supply data and a measure of error, and training searches for the settings that make the error small. This lets software handle problems nobody can fully specify. It also changes the contract: **a model is a statistical summary of past data, so it is sometimes wrong, and it decays when the world changes.** Every later design in this course, from guardrails to evals, exists to manage those two facts.

> *Before: nothing. Teaches: the AI family tree, the three kinds of learning, parameters versus hyperparameters, training versus inference, generalization, drift, and when not to use ML. Used by: every later lesson. AI02 L4 applies training at huge scale, and AI06 L8 turns "sometimes wrong" into hooks and gates.*

The hook: the rule that stopped working

You run payments at a bank. Fraud losses are rising, so someone writes the obvious rule:

```
boolean suspicious = txn.amount().compareTo(new BigDecimal("5000")) > 0;
```

In the first year it works well. On synthetic data shaped like this story, it's right about 97% of the time. Then fraudsters adapt, and legitimate spending patterns shift too. Fraud now clusters around 2,500 rather than 8,000.

Year	Rule	Accuracy on synthetic data
Year 1	Hand-written, flag above 5,000	96.6%
Year 2	Same rule, unchanged	**50.0%**

An accuracy of 50% on this balanced data is what a coin flip achieves. The rule now flags almost nothing. No exception was thrown, no deployment happened, and every dashboard stayed green.

> *Key idea*
>
> **The code was correct. The world moved.** In ordinary software, a bug is the code disagreeing with its specification. Here, there was no specification to disagree with, only an assumption about fraud that quietly stopped being true.

This failure raises three questions, which this lesson answers. Where should the 5,000 come from? How do you know when it's stale? And should a model or a rule decide this at all?

Intuition: an apprentice and a rulebook

Think of two ways to train a new fraud analyst.

The **rulebook** approach gives them a list: flag anything above 5,000; flag anything from these countries. They apply it exactly, and it fails as soon as a case isn't covered.

The **apprentice** approach gives them 100,000 past cases, each marked fraud or not. They begin to recognize patterns nobody ever wrote down. They can handle new cases, and they can also be confidently wrong, especially on cases unlike anything they've seen.

Traditional software is the rulebook. Machine learning is the apprentice. The Java engineer's instinct is to want the rulebook, and for many problems that instinct is right, as we'll see.

The family tree

These words get used interchangeably, but they are nested categories.

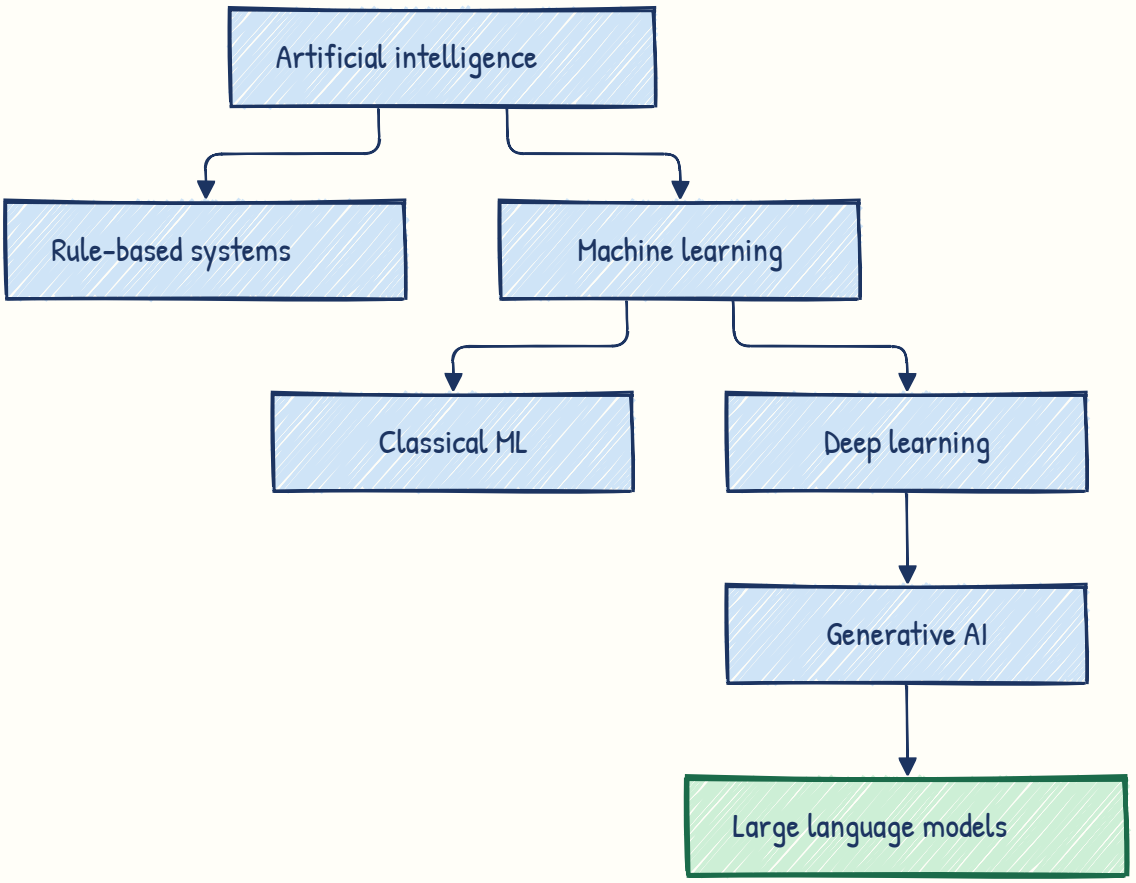


Figure 1. Each term is a narrower subset of the one above it. LLMs such as Claude sit at the bottom.

Term	Plain meaning	Example
Artificial intelligence	The broad goal	A chess engine, a fraud model, Claude
Rule-based system	AI with hand-written logic	A Drools rule set
Machine learning (ML)	Learns its behavior from data	A fraud model trained on past cases
Deep learning	ML using neural networks with many layers	Speech recognition
Generative AI	Models producing new text, images or audio	Image generators
Large language model (LLM)	Generative model trained to predict text	Claude

So a rule engine counts as AI, and it isn't ML. A gradient-boosted fraud model is ML, and it isn't deep learning. Claude is all of the above except rule-based.

The core mechanism: learning is a search

Strip away the mathematics, and all ML follows this loop.

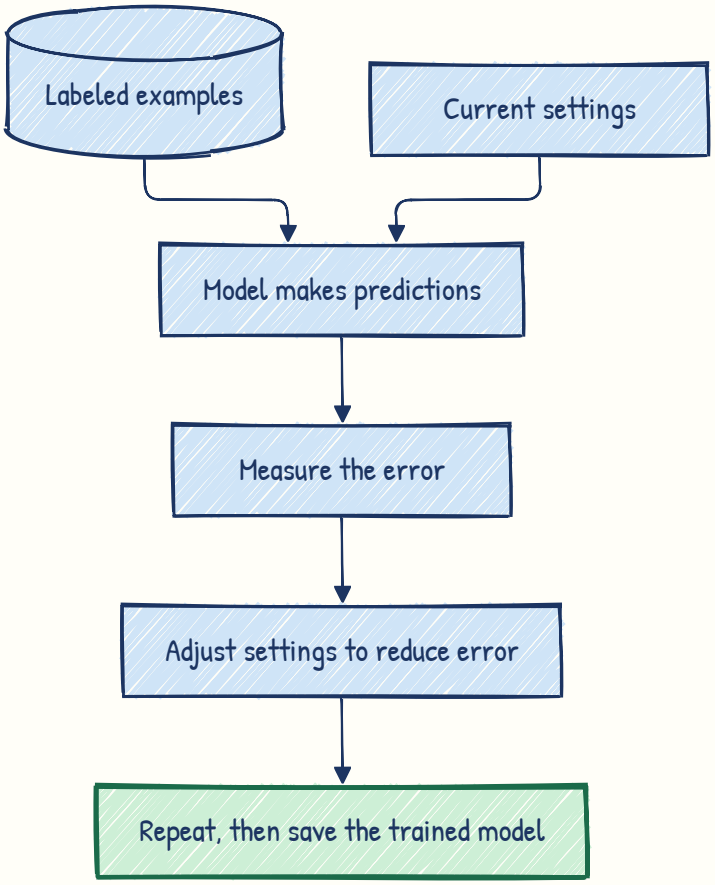


Figure 2. The training loop. Everything from a one-number threshold to Claude's pre-training uses this shape.

Here's the smallest real example. Our "model" is one number, a threshold. "Training" tries many values, and keeps the one with the fewest errors:

```java
static double learnThreshold(List<Txn> txns) {
    double best = 0;
    double bestAccuracy = -1;
    for (double candidate = 0; candidate <= 20_000; candidate += 50) {
        double score = accuracy(txns, candidate);
        if (score > bestAccuracy) {
            bestAccuracy = score;
            best = candidate;
        }
    }
    return best;
}
```

That is genuine machine learning, in the same way that bubble sort is genuine sorting. Real output, from `code/ai01-ml/FraudThreshold.java`, on synthetic data:

```
year 1: hand rule (5000) accuracy = 0.966
year 1: learned threshold = 3300, accuracy = 0.995
year 2: hand rule (5000) accuracy = 0.500
year 2: old learned threshold accuracy = 0.546
year 2: retrained threshold = 1250, accuracy = 0.985
```

Read those five lines carefully, because they contain the whole lesson:

1. **The data found a better value than the human did** (3,300 versus 5,000).
2. **Learning doesn't prevent decay.** The learned threshold also collapsed in year 2, reaching only 54.6%.
3. **What learning gives you is a cheap fix.** Rerun the same code on fresh data, and accuracy returns to 98.5%. Nobody had to rewrite anything.

> *Key idea*
>
> **ML doesn't make a system immune to change. It makes adapting to change a data problem rather than a code problem.** That is only useful if you have fresh labels, and a way to notice that adaptation is needed.

Real models don't scan values one by one. A model with millions of settings can't be searched that way, so AI01 L3 introduces gradient descent, which uses calculus to find a good direction to move. The loop remains the same.

The vocabulary you'll use forever

Term	Meaning	In the fraud example	Java analogy
Example	One row of data	One transaction	A row in a table
Feature	An input signal	The amount	A field on a DTO
Label	The known right answer	Was it fraud?	The expected value in a test
Parameter	A value that training learns	The threshold, 3,300	A config value, but written by the training job
Hyperparameter	A setting you choose before training	The 50 step size	A build setting
Training	Searching for good parameters	Running `learnThreshold`	A batch job producing an artifact
Inference	Using the trained model	Scoring a new transaction	A service method call
Model	Parameters plus the code that uses them	The number and the comparison	A JAR plus its config

The distinction between **parameter** and **hyperparameter** comes up constantly. Training changes parameters. You set hyperparameters, and only evaluation tells you whether you chose well. When people say a model has "billions of parameters", they mean billions of learned numbers.

The three ways to learn

Type	What you provide	What it learns	Example
Supervised	Inputs with correct answers	Predict the answer for new inputs	Fraud detection, ticket routing
Unsupervised	Inputs only	Structure in the data	Grouping customers into segments
Reinforcement	An environment and rewards	Actions that earn reward	Game play, and the RLHF stage of LLM training (AI02 L4)

Two more terms appear in LLM discussions:

→ **Self-supervised learning** creates labels from the data itself. Hide the next word in a sentence, and the true next word becomes the label. That's how an LLM can learn from raw text without humans labeling anything (AI02 L4).

→ **Transfer learning** starts from a model trained on one task and adapts it to another. It's the idea behind fine-tuning (AI11 L1).

Why it works at all: generalization

A model that merely memorized past transactions would be useless, because future transactions are different. What we want is **generalization**: good performance on data the model has never seen.

Generalization rests on an assumption so common that it's easy to miss:

> *The assumption underneath all of ML*
>
> **The future looks statistically like the training data.** When it doesn't, the guarantee disappears, and usually nothing tells you. The model keeps returning confident answers.

That is what broke in year 2. The technical name is **drift**.

Type of drift	What changed	Banking example
Data drift	The inputs	Average transaction size falls after a new product launch
Concept drift	The relationship between inputs and answer	Fraudsters move to small amounts, so "small" no longer means "safe"
Label drift	How often each answer occurs	Fraud rate doubles during a holiday

Drift also explains a design rule you'll meet again: **every ML system needs monitoring that can detect decay without waiting for a customer complaint.** For LLM systems, the form is different, but the need is the same (AI10 L3, AI10 L4).

What goes wrong by design

An ordinary service fails with an exception. A model fails by being wrong, while returning a perfectly valid response.

Property	Ordinary Java service	ML model
Correctness	Matches a specification	Measured statistically
A failure looks like	An exception, a 500	A normal response with a wrong answer
Same input twice	Same output, usually	Same for a fixed classical model. LLMs can vary (AI02 L5)
Debugging	Read the code	Inspect the data and errors
Testing	Assert exact outputs	Measure performance on a labeled set
Decay	Stable until someone changes it	Degrades as the world changes

This is the idea that drives the exam

Because models are sometimes wrong, a rule that must **always** hold can't depend on one. If compliance says "never refund above $500 without approval", you enforce it in code. **A model's 99.9% is not a guarantee, and 0.1% of a million transactions is a thousand incidents.** The exam returns to this in Task Statements 1.4 and 1.5, and AI06 L8 builds it.

Why the obvious alternatives fall short

Option	Works when	Falls short when
Hand-written rules	Logic is known, stable and must be auditable	Patterns are too complex, or they keep changing
More and more rules	Early on	Rules start interacting, and nobody dares delete any
Humans review everything	Volume is low and stakes are high	Volume grows; humans get expensive, slow and inconsistent
ML alone	Errors are tolerable and you have data	A rule has to be guaranteed or explained
Rules + ML + humans	Most real systems	It's more to build and operate

Google's Rules of Machine Learning opens with advice many teams need to hear. Rule 1 is "Don't be afraid to launch a product without machine learning." Rule 3 is the other half: "Choose machine learning over a complex heuristic." In other words, start with simple rules, and move to ML once they become hard to maintain and you have the data.

Should this be ML at all?

Ask these in order, and stop at the first "no".

#	Question	Stop here if	Choose
1	Can you write the rules, and do they stay stable?	Yes	Plain code or a rule engine
2	Do you have data, or a way to get it?	No	Rules, or a pre-trained model
3	Can the business tolerate some errors?	No	ML assists, while code or a person decides
4	All three passed	—	Use ML, and monitor it

Every early exit is a good outcome, because it leaves you with simpler software.

The second exit matters more than it used to. Calling Claude means using a model someone else trained. You can often solve a language problem with **no training data**, only a good prompt and an evaluation set. That changes the economics, and it's one reason LLMs spread so quickly.

How the field solved it

Anthropic didn't invent these ideas. They come from decades of research across the field. This course says so where that's the case, and Anthropic's own work starts appearing in AI02.

The problem	How the field addressed it
Rules can't capture messy patterns	Learn parameters from labeled data
Models memorize instead of generalizing	Hold out unseen test data (AI01 L2)
Simple models can't capture complex patterns	Multi-layer neural networks, trained by backpropagation (AI01 L3)
Deep networks need vast data and compute	GPUs and large datasets. AlexNet in 2012 is the usual marker of the turning point
Labeling data by hand doesn't scale	Self-supervised learning from raw text (AI02 L4)

In the real world

Anthropic's documentation carries this statistical mindset into LLM work. Its guide to developing tests asks you to define success criteria that are specific, measurable, achievable and relevant, to include edge cases, and to automate grading where possible. That is the same discipline as measuring a fraud model on a labeled set, applied to a prompt. AI10 L3 covers it in depth.

The production architecture

No company's internal fraud system is being described here. **This is a recommended reference design, built to teach the pattern.** For the full ML lifecycle, see HLD M21C, which covers feature stores and training pipelines. This section focuses on one question: how do rules, models and people share a decision?

Baseline: rules in the service. Easy to audit, but every change is a deployment, and the rules decay silently, as we saw.

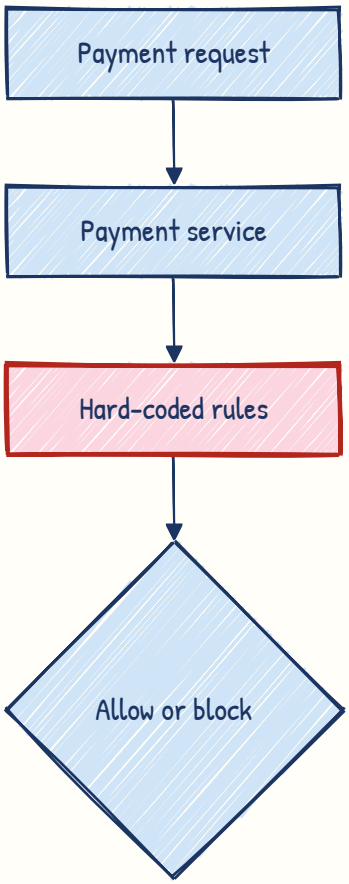


Figure 3. Baseline. Logic lives in the application, so fraud knowledge changes only when code ships.

Improved: the model decides. A scoring service adds pattern recognition, and a retraining job handles adaptation. This creates a new problem: **now nothing guarantees the rules that must never break**, and nobody can review the cases where the model is unsure.

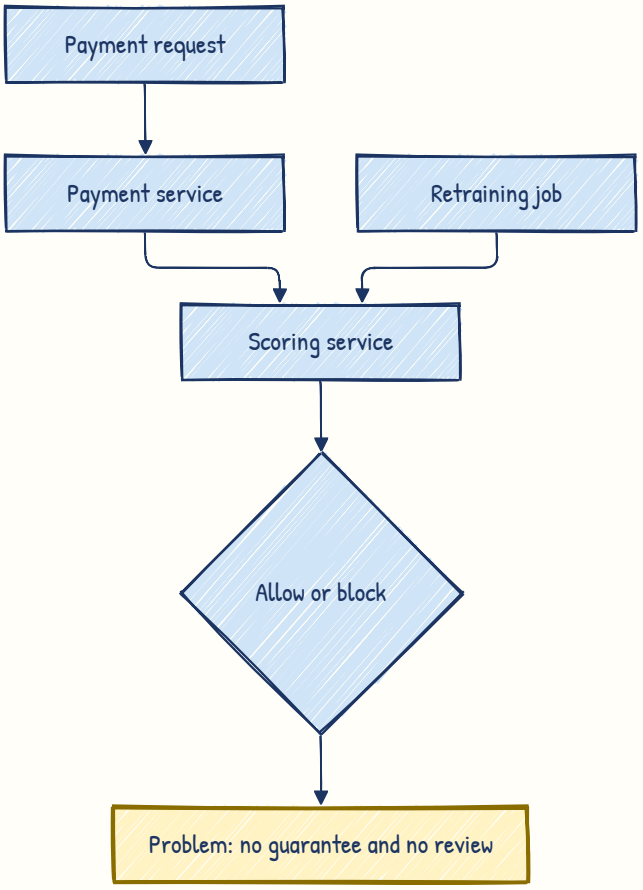


Figure 4. Improved. The model adapts to change, but hard limits and uncertain cases now have no owner.

Production: each decision goes to the right owner. Hard limits run first in deterministic code. The model handles the patterns. Uncertain cases go to people, and their decisions become new labels.

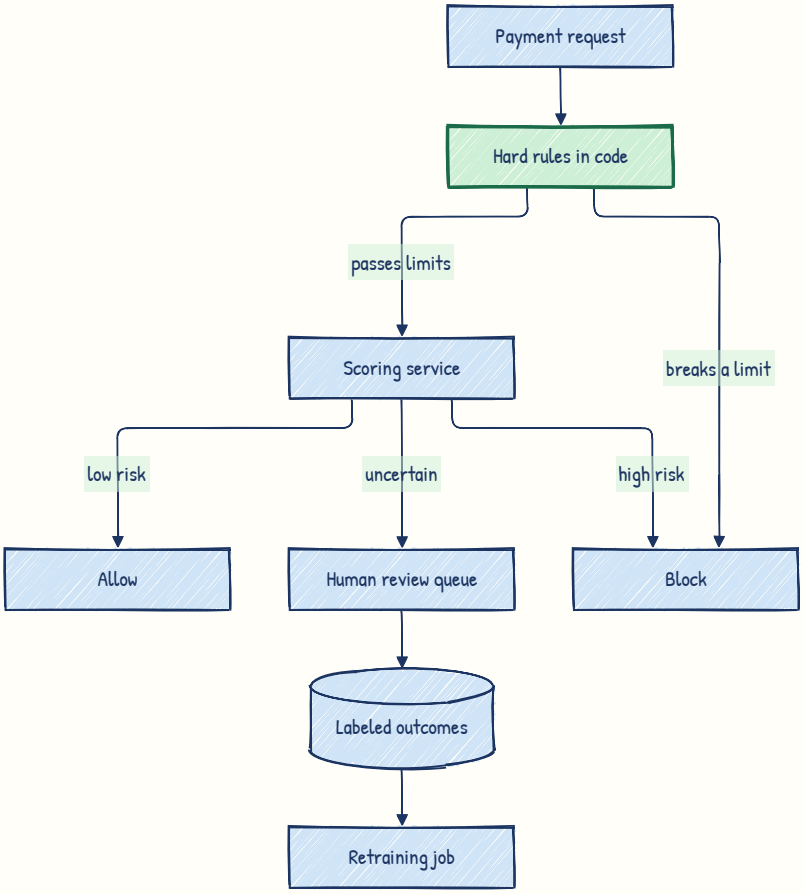


Figure 5. Production. Code owns the guarantees, the model owns the patterns, and people own the uncertainty. Review outcomes feed the next training run.

Component	Why it exists	How it communicates
Hard rules	Legal and business limits must always hold	In-process, before any model call
Scoring service	Finds patterns rules can't express	Synchronous call with a tight timeout
Review queue	Humans handle what the model can't decide	Asynchronous, using Kafka or a work queue
Labeled outcomes	Training needs ground truth, and fraud labels arrive late	Written by review tools and chargeback feeds
Retraining job	Adapts to drift	Scheduled batch job that publishes a versioned model

What fails, and how it's contained

Failure	Effect	Containment
Scoring service times out	No score	Fall back to rules, and choose whether to fail open or closed for each transaction type
Silent drift	Quality falls with no errors	Monitor score distributions and label-based precision
Bad model released	Mass false blocks	Canary release, plus an instant rollback to the previous version
Late labels	Recent fraud is invisible to training	Measure on matured data, and track the labeling delay
Feedback loop	Blocked transactions never get a true label	Allow a tiny audited sample through, or use review outcomes

The same shape returns when the "model" is Claude. In AI06 L9, hooks enforce limits, the agent handles judgment, and `escalate_to_human` handles uncertainty.

Trade-offs

Decision	You gain	It costs	Choose it when
Rules	Exact, explainable, no data needed	Brittle and manual	Logic is known and stable
Classical ML	Adapts, handles many signals	Needs labels and monitoring	Tabular data with history
LLM via API	No training data, handles language	Cost per call, varying output	The input is unstructured language
Human review	Judgment and accountability	Slow and expensive	Stakes are high and cases are ambiguous

When not to use ML: when a rule is known and must always hold, when you can't tolerate errors, when you have no way to measure quality, or when a lookup table would do the job.

Common mistakes

1. **Treating model output as fact.** It's a statistical estimate. Design for it being wrong.
2. **Enforcing compliance with a model.** Hard limits belong in code.
3. **Measuring only on launch day.** Drift arrives later, without any error.
4. **Using ML because it's fashionable.** If `if (amount > limit)` solves the problem, ship that.
5. **Assuming retraining happens by itself.** It needs fresh labels, and fraud labels can take weeks.
6. **Confusing parameters and hyperparameters.** Training sets the first, you set the second.

What to say in an interview

When asked "would you use ML here?", don't start with an algorithm. Say: "First, can it be written as stable rules? If so, I'd use code. If not, I'd check for data, and ask what a wrong answer costs. I'd keep hard limits in code, use a model for patterns, and route uncertain cases to people." That shows you treat ML as an engineering choice.

Certification lens

How this lesson maps to CCAR-F

ML theory is **not on the exam**. But the idea that model behavior is probabilistic underlies **TS 1.4** and **TS 1.5**, where programmatic enforcement beats prompt instructions. It also underlies the answer pattern in the guide's first sample question.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Customer Support Resolution Agent · Medium. A refund agent follows the "no refunds above $500 without approval" instruction in 99.7% of cases. Finance requires the limit to hold in all cases. What's the most appropriate change?

A. Move the instruction to the top of the system prompt, and repeat it at the end.

B. Add a tool-call interception hook that blocks refunds above $500 and redirects them to escalation.

C. Have the agent state its confidence, and escalate when confidence is low.

D. Switch to a more capable model and re-measure.

A **B.** A guaranteed rule needs deterministic enforcement (TS 1.5). A and D may raise compliance but still rely on probabilistic behavior. C measures the model's opinion of itself rather than the actual amount, so it can't guarantee anything.

Q Q2 · Concept · Easy. Your learned threshold's accuracy fell from 99.5% to 54.6% with no code change. Name the problem, and the fix.

A Drift: the live data no longer resembles the training data. Here it's concept drift, since fraud moved to smaller amounts. Detect it with monitoring, and retrain on fresh labeled data. In the demo, retraining restored 98.5%.

Q Q3 · Design decision · Hard. A product manager asks you to "use AI" to apply a published tax table to invoices. What do you recommend, and why?

A Plain code. The rule is known, stable, must be exact and must be auditable. A model adds cost and a non-zero error rate, and it has no benefit here. AI could help elsewhere, such as extracting the invoice fields from a PDF. The tax calculation itself should remain deterministic.

Revision - Lesson 1

→ AI is the broad goal. ML learns from data. Deep learning uses many-layer networks. LLMs are generative deep learning models.

→ Training searches for parameters that reduce error on examples. Inference uses the result.

→ Parameters are learned, and hyperparameters are chosen by you.

→ Supervised learning has labels, unsupervised finds structure, and reinforcement learns from rewards. Self-supervised learning makes labels from the data itself.

→ Generalization assumes the future resembles the training data. Drift breaks that assumption silently.

→ ML failures are wrong answers in valid responses, so measure statistically and monitor.

→ ML makes adapting to change a data problem, provided you have fresh labels.

→ Production pattern: code owns guarantees, models own patterns, and people own uncertainty.

→ Try rules first. Use ML when the rules become unmanageable and you have data or a pre-trained model.

Your notes